```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,classification_report

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

import kagglehub

# Download latest version
path = kagglehub.dataset_download("akshaydattatraykhare/diabetes-dataset")

print("Path to dataset files:", path)

import pandas as pd
import os
df=pd.read_csv(os.path.join(path,'diabetes.csv'))


df

print(df.head())


print(df.tail())


print(df.describe())


print(df.info())
```

```
print(df.shape)


print(df.isnull().sum())

df['Outcome'].value_counts()        # from this we can check the chances of getting accuracy

sns.heatmap(df.corr(),annot=True)
plt.show()

sns.boxplot(x='Outcome',y='Glucose',data=df)
plt.show()

sns.boxplot(x='Outcome',y='Age',data=df)
plt.show()

sns.countplot(x='Outcome',data=df)
plt.title("dibetes distribution")
plt.show()

sns.histplot(df['BMI'], kde=True)
plt.title("BMI distribution")
plt.show()

sns.pairplot(df, hue='Outcome')
plt.show()

Q1=df.quantile(0.25)
Q3=df.quantile(0.75)
IQR=Q3-Q1

df=df[~((df<(Q1-1.5*IQR))|(df>(Q3+1.5*IQR))).any(axis=1)]

X=df.drop('Outcome',axis=1)
y=df['Outcome']

X

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

from imblearn.over_sampling import SMOTE
sm=SMOTE(random_state=42)
X_train,y_train=sm.fit_resample(X_train,y_train)
```

```
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
from sklearn.tree import DecisionTreeClassifier


models={
   'Logestic Regression':LogisticRegression(),
   'Decision Tree':DecisionTreeClassifier(),
   'Random forest':RandomForestClassifier(),
   'SVM':SVC(),
   'KNN':KNeighborsClassifier()
}


from sklearn.metrics import precision_score, recall_score, f1_score

results=[]
for name, model in models.items():
  model.fit(X_train,y_train)
  y_pred=model.predict(X_test)
  acc=accuracy_score(y_test,y_pred)
  prec=precision_score(y_test,y_pred)
  rec=recall_score(y_test,y_pred)
  f1=f1_score(y_test,y_pred)
  results.append({
      'Model':name,
      'Accuracy': acc,
      'Precision':prec,
      'Recall':rec,
      'F1 Score':f1
  })

results_df=pd.DataFrame(results)
print(results_df)

plt.bar(results_df['Model'],results_df['Accuracy'])
plt.title("model acuracycomparison")
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.xticks(rotation=45)
```

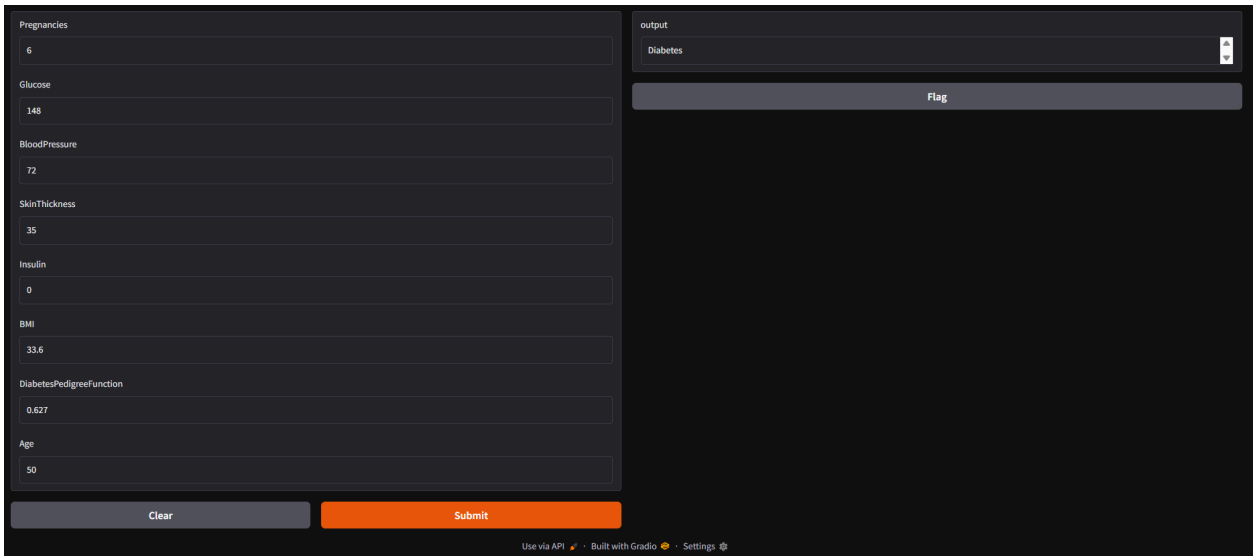
```
plt.show()

import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
plt.figure(figsize=(8, 6))
for model_name, model in models.items():
    #TRAIN THE MODEL
    model.fit(X_train, y_train)
    if hasattr(model, 'predict_proba'):
        y_prob = model.predict_proba(X_test)[:, 1]
    else:

        y_prob = model.decision_function(X_test)

    #COMPUTE ROC
    fpr, tpr, thresholds = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)
    #plot
    plt.plot(fpr, tpr, label=f"{model_name} (AUC = {roc_auc:.2f})")

plt.plot([0,1],[0,1],'k--')
plt.xlabel("False positive Rate")
plt.ylabel("True positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

best_model_name=results_df.loc[results_df['Accuracy'].idxmax(),'Model']
print("best model: ",best_model_name)

best_model=models[best_model_name]
best_model.fit(X_train,y_train)
pred_rf=best_model.predict(X_test)

cm=confusion_matrix(y_test,pred_rf)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("confusion matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()

print(classification_report(y_test,pred_rf))
```

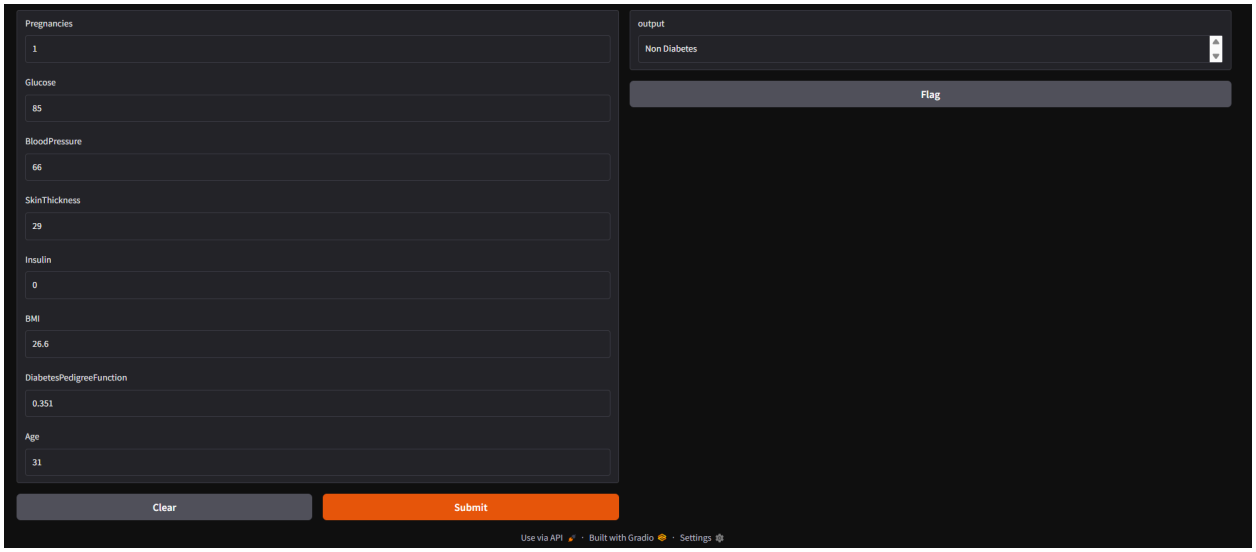
USING STRATIFY

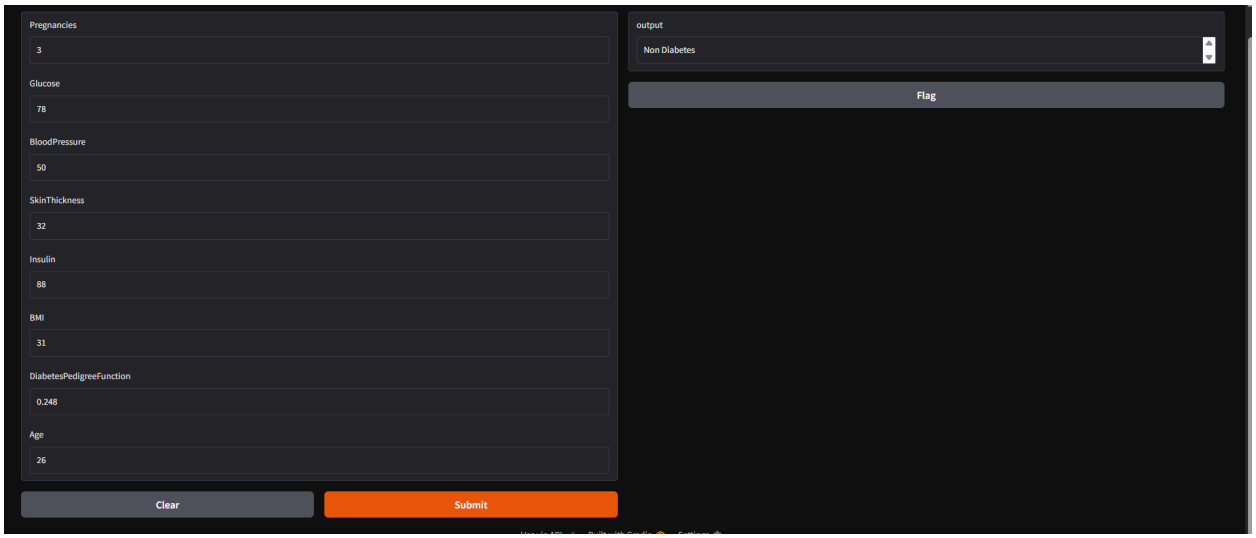
```
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
from sklearn.tree import DecisionTreeClassifier


models={
   'Logestic Regression':LogisticRegression(),
   'Decision Tree':DecisionTreeClassifier(),
   'Random forest':RandomForestClassifier(),
   'SVM':SVC(),
   'KNN':KNeighborsClassifier()
}


from sklearn.metrics import precision_score, recall_score, f1_score

results=[]
for name, model in models.items():
  model.fit(X_train,y_train)
  y_pred=model.predict(X_test)
  acc=accuracy_score(y_test,y_pred)
  prec=precision_score(y_test,y_pred)
  rec=recall_score(y_test,y_pred)
  f1=f1_score(y_test,y_pred)
  results.append({
      'Model':name,
      'Accuracy': acc,
      'Precision':prec,
      'Recall':rec,
      'F1 Score':f1
  })

results_df=pd.DataFrame(results)
print(results_df)

import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
plt.figure(figsize=(8, 6))
```

```
for model_name, model in models.items():
    #TRAIN THE MODEL
    model.fit(X_train, y_train)
    if hasattr(model, 'predict_proba'):
        y_prob = model.predict_proba(X_test)[:, 1]
    else:

        y_prob = model.decision_function(X_test)

    #COMPUTE ROC
    fpr, tpr, thresholds = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)
    #plot
    plt.plot(fpr, tpr, label=f"{model_name} (AUC = {roc_auc:.2f})")

plt.plot([0,1],[0,1],'k--')
plt.xlabel("False positive Rate")
plt.ylabel("True positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

best_model_name=results_df.loc[results_df['Accuracy'].idxmax(),'Model']
print("best model: ",best_model_name)

best_model=models[best_model_name]
best_model.fit(X_train,y_train)
pred_rf=best_model.predict(X_test)

cm=confusion_matrix(y_test,pred_rf)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("confusion matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()

print(classification_report(y_test,pred_rf))

import pickle
pickle.dump(best_model,open('diabetes_model.pkl','wb'))
pickle.dump(scaler,open('scaler.pkl','wb'))

import gradio as gr
import numpy as np
```

```
import pickle

model = pickle.load(open('diabetes_model.pkl','rb'))
scaler = pickle.load(open('scaler.pkl','rb'))

def predict_diabetes(Pregnancies,Glucose,BloodPressure,SkinThickness,
                     Insulin,BMI,DiabetesPedigreeFunction,Age):

    data = np.array([[Pregnancies,Glucose,BloodPressure,SkinThickness,
                      Insulin,BMI,DiabetesPedigreeFunction,Age]])


    data = scaler.transform(data)

    result = model.predict(data)

    return 'Diabetes' if result[0]==1 else 'Non Diabetes'

demo = gr.Interface(
    fn=predict_diabetes,
    inputs=['number','number','number','number','number','number','number','number'],
    outputs='text',
    title='Diabetes Prediction System'
)

demo.launch(share=True)
```




Pregnancies
1
Glucose
85
BloodPressure
66
SkinThickness
29
Insulin
0
BMI
26.6
DiabetesPedigreeFunction
0.351
Age
31
Clear
Submit
output
Non Diabetes
Flag
Use via API · Built with Gradio · Settings


Pregnancies
3
Glucose
78
BloodPressure
50
SkinThickness
32
Insulin
88
BMI
31
DiabetesPedigreeFunction
0.248
Age
26
Clear
Submit
output
Non Diabetes
Flag